Templated ID Generation Service — Architecture and Implementation Plan

Nov 13, 2026 · Nazaar

A Quarkus + Postgres service that issues formatted government IDs (receipts, licences, permits) from configurable templates for 10 city tenants, with zero duplicates under parallel load. This document is the input for building it with Claude Code: the sections up to the ADRs are the specification, and **Implementation plan** gives the order of work with a done-check for each step.

Context and goals

City services need human-readable IDs such as `PG-AMR-2026-27-000431`. Templates come from `data/templates.json` and tenants from `data/cities.csv`; both are provided and must not be edited.

Hard rule: an ID is never issued twice. Gaps in a sequence are acceptable.

Goals

- Issue one ID, or `count` IDs in one call, for a (template, city).
- Zero duplicates under parallel single and bulk calls, across several backend instances.
- Structured errors (never stack traces) for an unknown template, an unknown city, or a named counter with no configuration.
- Thin web UI: pick template + city, enter a count, issue, see the results and the recently issued IDs.
- `docker compose up` is the only command needed: build, start Postgres + backend + UI, load the data.
- A concurrency test that runs genuinely parallel workers against the running app.

Non-goals: authentication, editing templates at runtime, cancelling or reusing IDs, audit history, multi-region.

Worked example: `PG_RECEIPT` for `pb.amritsar` on 13 Nov 2026.

Placeholder	Resolves to	Rule
`[CITY]`	`AMR`	city_code for tenant `pb.amritsar`

Placeholder	Resolves to	Rule
`[FY:yyyy-yy]`	`2026-27`	Nov 2026 is in the Apr 2026–Mar 2027 financial year
`[SEQ:6]`	`000001`	first value of the (PG_RECEIPT, AMR, 2026-27) counter, padded to 6

Result: `PG-AMR-2026-27-000001`, then `…-000002`. Ludhiana counts separately from `PG-LDH-2026-27-000001`.

Templates and placeholder grammar

A template is parsed once at startup into a list of tokens (literal text and placeholders). Every request then renders that list: there is no string replacement at request time.

Placeholder	Renders	Rule
`[CITY]`	`AMR`	`city_code` of the resolved tenant
`[FY:yyyy-yy]`	`2026-27`	Indian financial year, Apr–Mar: start year = year if month ⩾ April, else year − 1
`[FY:yy-yy]`	`26-27`	Same, two-digit years
`[CY:fmt]`	`2026`, `11-2026`	Current date formatted with `fmt` as a `java.time` pattern (`yyyy`, `MM-yyyy`)
`[SEQ:n]`	`000001`	Next value of the template's implicit counter, zero-padded to at least `n` digits
`[SEQ_NAME:n]`	`1000`	Next value of named counter `SEQ_NAME` (config in `counters`), padded the same way

Rendering rules

- **Clock:** `Asia/Kolkata`, read once per request, so every ID in a bulk call shares the same date and period.
- **Padding is a minimum width.** A value wider than `n` is printed in full, never truncated (truncation would wrap around and duplicate). Log a warning the first time a key overflows.
- **Literal text** (`-`, `/`, prefixes) is copied unchanged.

- **Unknown placeholder syntax** fails startup with the template ID and position. A named counter with no `counters` entry is not a startup error: the template loads, and issuing from it returns 422 `COUNTER_NOT_CONFIGURED`.
- **`start`** is the first value issued for a named counter; implicit counters start at 1.

The six provided templates

Template	Format	Counter scope (ADR-003)	First ID for Amritsar	Note
PG_RECEIPT	`PG-[CITY]-[FY:yyyy-yy]-[SEQ:6]`	template + city + FY	`PG-AMR-2026-27-000001`	Resets each FY
TL_LICENSE	`TL/[CITY]/[CY:yyyy]/[SEQ:5]`	template + city + year	`TL/AMR/2026/00001`	`/` separators; resets each year
PGR_COMPLAINT	`PGR-[SEQ:8]`	template only (global)	`PGR-00000001`	No city in the output, so one counter for all cities
WS_CONNECTION	`WS-[CITY]-[SEQ_WS:4]-[CY:MM-yyyy]`	`SEQ_WS` + city + month	`WS-AMR-1000-11-2026`	Start 1000; width 4 grows past 9999
BPA_PERMIT	`BPA-[CITY]-[FY:yy-yy]-[SEQ_BPA_PERMIT:6]`	`SEQ_BPA_PERMIT` + city + FY	`BPA-AMR-26-27-000001`	Short FY form
EST_ALLOTMENT	`EST-[CITY]-[SEQ_EST_ALLOT:5]`	—	422 `COUNTER_NOT_CONFIGURED`	`SEQ_EST_ALLOT` has no `counters` entry

Tenants: 10 cities across Punjab, Karnataka, Maharashtra and Tamil Nadu. A request may name a city by tenant ID (`pb.amritsar`) or city code (`AMR`).

Acceptance criteria

The build is done when every box below can be ticked from a fresh clone. Reviewers will run `docker compose up` and send parallel requests to the endpoints.

Functional

- [ ] `POST /api/ids` returns one ID for every provided template × city except EST_ALLOTMENT.
- [ ] `POST /api/ids/bulk` returns exactly `count` distinct IDs, in one response.
- [ ] `city` accepts `pb.amritsar`, `AMR` and `amr`.
- [ ] Unknown template → 404 `UNKNOWN_TEMPLATE`; unknown city → 404 `UNKNOWN_CITY`; EST_ALLOTMENT → 422 `COUNTER_NOT_CONFIGURED`; `count` outside 1–1000 → 400 `INVALID_COUNT`. Every error uses the same JSON shape, with no stack traces.

- [ ] The UI issues single and bulk IDs and shows a recent-IDs list.

Concurrency and operations

- [ ] The load test (50 parallel workers, mixed single + bulk, all templates and cities, 3 backend instances) reports 0 duplicates.
- [ ] A backend instance restarted mid-test still gives 0 duplicates.
- [ ] `docker compose up` from a fresh clone is the only command; the README gives the URL.
- [ ] `data/` files are byte-for-byte unchanged.

Documents in the repo

- [ ] `spec.md`, `plan.md`, `README.md`, `docs/ai-log.md`, the Postman collection `egov-poc-id-gen.postman_collection.json`, and `docs/adr/0001–0006`.

Decisions on the brief's open points

The brief is deliberately underspecified. Each decision below follows one principle: **when in doubt, choose the option that cannot produce a duplicate.** These go into `spec.md`, and the ones that need confirming are in the questions table at the end.

#	Open point	Decision	Why
1	`PGR-[SEQ:8]` has no `[CITY]`, but `[SEQ]` is scoped per (template, city)	One global counter for templates whose output has no `[CITY]`	Per city, every city would emit `PGR-00000001`
2	`SEQ_EST_ALLOT` has no `counters` entry	422 `COUNTER_NOT_CONFIGURED`; never auto-create it	Required by the brief
3	`[SEQ_WS:4]` starts at 1000, so it holds only 9,000 values	`n` is a minimum width; the ID grows; never truncate	Truncation wraps around into duplicates
4	Do counters reset with the FY / year / month?	Yes, but only when that period is printed in the ID: a new period means a new sequence	`TL/AMR/2026/00001` and `TL/AMR/2027/00001` differ; `PGR-` has no period, so it never resets

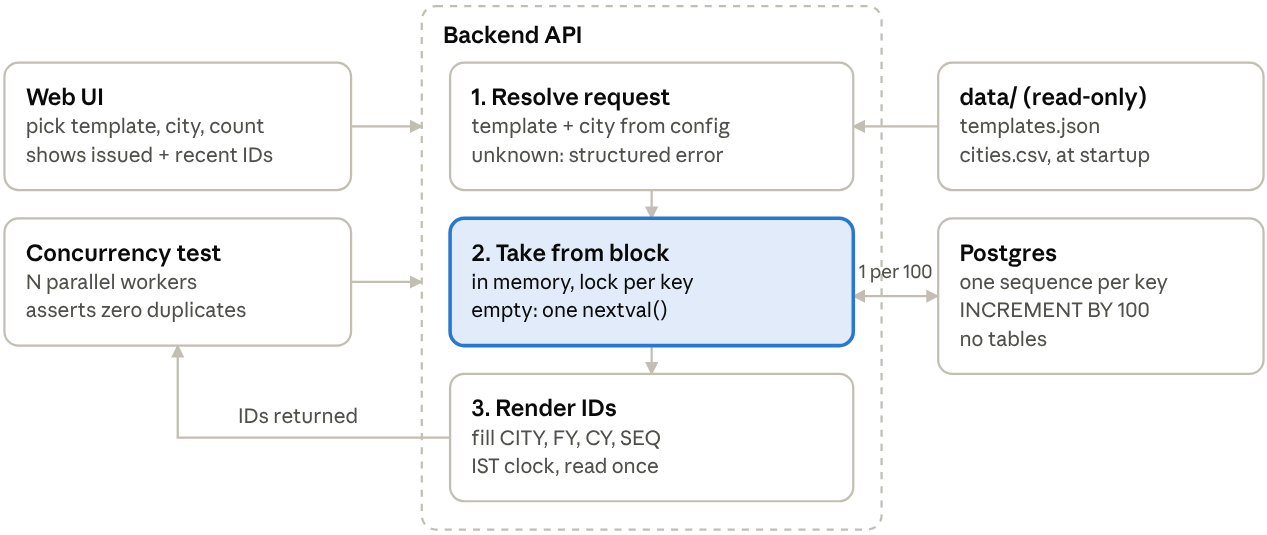
#	Open point	Decision	Why
5	Which clock decides FY/CY?	Asia/Kolkata, read once per request	31 Mar 20:00 UTC is already 1 Apr IST
6	Meaning of `start`	The first issued value equals `start`	Most natural reading
7	`city` = tenant ID or city code	Accept both, case-insensitive; respond with the canonical values	The brief allows either
8	Bulk limits	`count` 1–1000; all-or-nothing response	Bounds the response size; on failure, gaps are acceptable
9	Multiple instances and restarts	Postgres sequences own the high-water mark; instances cache only blocks they took via `nextval` (ADR-001)	Counters held only in memory would duplicate across instances
10	A named counter used by two templates	One shared sequence per (counter, city, period)	Matches "named counter scoped per city"

Out of scope: authentication, editing templates at runtime, cancelling or reusing IDs, audit history, multi-region.

Architecture

The whole problem comes down to one atomic step: a Postgres sequence hands out each block of 100 exactly once, and the app issues from that block in memory.

IDs come from an in-memory block; Postgres is hit once per 100



suggested architecture · 3 containers, 1 atomic step

The core of the solution is one Postgres sequence per counter key, incrementing by 100. Each instance calls nextval only when its in-memory block runs out (ADR-001):

```
CREATE SEQUENCE IF NOT EXISTS seq_pg_receipt_amr_2026_27 START WITH 1
INCREMENT BY 100;  -- once, at startup
SELECT nextval('seq_pg_receipt_amr_2026_27');  -- returns the block start: 1,
then 101, 201…
```

`nextval` is atomic and never blocks other callers, so every instance gets a disjoint block `[base, base+100)`. Different keys never contend with each other. This is the classic hi-lo pattern used for voucher numbers in core banking, here built on a plain Postgres sequence.

- **One sequence per key** = scope + period, e.g. `seq_pg_receipt_amr_2026_27`, `seq_pgr_complaint` (global), `seq_ws_amr`.
- **Not recommended:** in-process locks or `SELECT max()+1` (unsafe with more than one replica), or Redis INCR (adds a second store for no gain at this scale).
- **Stack:** Quarkus (JVM mode) + Postgres 16. The static UI is served by the same app (ADR-002).
- **Data model:** no tables. Postgres holds only sequences. Config is loaded from `data/` into memory, and the recent-IDs view is an in-memory list per instance.

- **Errors:** one shape, e.g. `{"error":{"code":"UNKNOWN_CITY","message":"…","details":{…}}}`, with 404 for an unknown template/city, 422 for an unconfigured counter, and 400 for a bad `count`.

Runtime topology (`docker-compose.yml`)

Service	Image	Exposed	Notes
`postgres`	`postgres:16-alpine`	internal	Healthcheck `pg_isready`; holds only sequences
`backend`	built from `Dockerfile` (tag `nazaarblue/idgen:1.0.0`)	internal :8080	`deploy.replicas: 3`; `depends_on: postgres (service_healthy)`; mounts `./data:/data:ro`
`nginx`	`nginx:alpine`	**host :8080**	Round-robins to `backend:8080`; the only public port, so reviewers' requests spread over 3 instances
`loadtest`	`python:3.12-slim`	—	`profiles: [test]`; runs `loadtest/loadtest.py` against `nginx`

API contract

All endpoints are JSON under `/api`. The UI is served from `/`.

Method	Path	Purpose
POST	`/api/ids`	Issue one ID
POST	`/api/ids/bulk`	Issue `count` IDs
GET	`/api/ids/recent?templateId=&city=&limit=50`	Recently issued IDs on the instance that serves the request
GET	`/api/templates`	Templates for the UI dropdown, including whether each can issue

Method	Path	Purpose
GET	`/api/cities`	Cities for the UI dropdown
GET	`/q/health/ready`	Readiness: config loaded and Postgres reachable

Issue one

```
POST /api/ids
{ "templateId": "PG_RECEIPT", "city": "pb.amritsar" }

200 OK
{ "id": "PG-AMR-2026-27-000001", "templateId": "PG_RECEIPT",
  "tenantId": "pb.amritsar", "cityCode": "AMR", "issuedAt": "2026-11-
13T05:12:04+05:30" }
```

Issue bulk

```
POST /api/ids/bulk
{ "templateId": "TL_LICENSE", "city": "LDH", "count": 3 }

200 OK
{ "ids": ["TL/LDH/2026/00001", "TL/LDH/2026/00002", "TL/LDH/2026/00003"],
  "count": 3, "templateId": "TL_LICENSE", "tenantId": "pb.ludhiana",
"cityCode": "LDH",
  "issuedAt": "2026-11-13T05:12:09+05:30" }
```

Errors: one shape for every failure.

```
{ "error": { "code": "COUNTER_NOT_CONFIGURED",
             "message": "Template EST_ALLOTMENT uses counter SEQ_EST_ALLOT,
which has no configuration",
             "details": { "templateId": "EST_ALLOTMENT", "counter":
"SEQ_EST_ALLOT" } } }
```

Code	HTTP	When
`VALIDATION_ERROR`	400	Body missing, malformed JSON, or a missing `templateId` / `city`
`INVALID_COUNT`	400	`count` not an integer in 1–1000

Code	HTTP	When
`UNKNOWN_TEMPLATE`	404	`templateId` not in `templates.json`
`UNKNOWN_CITY`	404	`city` matches no tenant ID or city code
`COUNTER_NOT_CONFIGURED`	422	Template uses a named counter with no `counters` entry
`SEQUENCE_UNAVAILABLE`	503	Postgres unreachable while refilling a block
`INTERNAL_ERROR`	500	Anything else; logged with a correlation ID, which is returned in `details`

Code structure

Java 25, Quarkus 3.x, Maven wrapper, one module. Package `org.egov.idgen`:

Package	Classes	Responsibility
`config`	`IdgenConfig` (`@ConfigMapping`), `DataLoader` (`@Startup`), `TemplateRegistry`, `CityRegistry`	Read `data/` once; validate; fail startup with file + line on bad input
`template`	`Token` (sealed: `Literal`, `City`, `FinancialYear`, `CalendarDate`, `Sequence`), `TemplateParser`, `CompiledTemplate`, `CounterKey`	Parse formats into tokens; derive the counter key; render one ID from (tokens, city, clock, value)
`sequence`	`SequenceNames`, `SequenceStore`, `BlockAllocator`, `Block`	Create sequences, call `nextval`, hand out values from in-memory blocks
`issue`	`IdService`	Resolve → key → allocate → render → record recent
`recent`	`RecentIds`	Bounded in-memory list (50) per (template, city)
`api`	`IdResource`, `MetaResource`, request/response `record`s, `ErrorCode`, `IdgenException`, `ErrorMapper`	REST endpoints; one `ExceptionMapper` that produces the error shape

Issue flow (`IdService.issue`)

```
Issued issue(String templateId, String city, int count) {
    var tpl  = templates.get(templateId);           // UNKNOWN_TEMPLATE
    var ten  = cities.resolve(city);                // UNKNOWN_CITY (tenant
id or code, case-insensitive)
    tpl.requireConfiguredCounter();                 //
COUNTER_NOT_CONFIGURED
    var now  = ZonedDateTime.now(config.zone());    // read once
    var key  = tpl.counterKey(ten, now);            // ADR-003 scoping rule
    long[] v = allocator.take(key, count);          // ADR-001
    var ids  = Arrays.stream(v).mapToObj(x -> tpl.render(ten, now,
x)).toList();
    recent.add(tpl.id(), ten.code(), ids);
    return new Issued(ids, tpl, ten, now);
}
```

Block allocator (`BlockAllocator.take`)

```
// one Block per counter key per instance; the lock is per key, never global
long[] take(CounterKey key, int count) {
    var block = blocks.computeIfAbsent(key, k -> new Block());
    synchronized (block) {
        long[] out = new long[count];
        for (int i = 0; i < count; i++) {
            if (block.next == block.end) {
                long base = store.nextval(key);        // creates the
sequence on first use
                block.next = base;
                block.end  = base + store.increment(key); // read from
pg_sequences, never from config
            }
            out[i] = block.next++;
        }
        return out;
    }
}
```

A bulk request that crosses a block boundary refills inside the loop. The cost is one `nextval` per 100 values; batching these into one `generate_series` call can come later.

Sequence store

- **Name:** `seq_` + key parts joined by `_`, lowercased, with anything outside `[a-z0-9_]` replaced by `_`. If it would exceed 63 characters, truncate and append an 8-character hash.
- **Create:** `CREATE SEQUENCE IF NOT EXISTS <name> START WITH <start> INCREMENT BY <block>`. Pre-create at startup for every key's current period, and create on first use otherwise. Treat SQLSTATE `23505` / `42P07` from a concurrent create as success.
- **Increment:** read `increment_by` from `pg_sequences` after create and cache it per key.
- **Client:** reactive PG client or Agroal JDBC; only `CREATE SEQUENCE`, `nextval` and the `pg_sequences` read. No ORM.

Tests

- **Unit (`TemplateParserTest`, `RenderTest`):** all six templates; FY on 31 Mar vs 1 Apr; padding growth (`SEQ_WS` value 10000); PGR key has no city; EST rejected.
- **API (`IdResourceTest`, RestAssured + Quarkus Dev Services Postgres):** each error code and its shape; tenant ID / code / lower-case all resolve.
- **Concurrency (`loadtest/loadtest.py`):** `multiprocessing.Pool(50)` with real OS processes; each worker loops over random (template, city, single or bulk 1–50) for N seconds; all IDs are collected and the test fails if `len(set(ids)) != len(ids)`. It prints totals, duplicates and requests/sec. Run with `docker compose --profile test run --rm loadtest`.
- **Throughput (`loadtest/idgen.jmx`, JMeter):** a thread group of 50 users for 60 s against Nginx, mixing `POST /api/ids` and `POST /api/ids/bulk` over random templates and cities from a CSV data set. Assertions: HTTP 200 and the `id` matches its template's pattern. Run headless with `jmeter -n -t loadtest/idgen.jmx -l loadtest/out/results.jtl -e -o loadtest/out/report`; the HTML report gives throughput, p95/p99 latency and error rate. JMeter measures speed; `loadtest.py` stays the proof of zero duplicates, because it collects every ID.

ADR-001: Postgres sequences with in-memory blocks, no tables

Decision: each counter key gets one Postgres sequence with `INCREMENT BY 100`. A backend instance calls `nextval` once, takes the 100 numbers starting there, and issues them from memory. The service has no tables. Accepted, 13 Nov 2026.

Context. If every ID needs a DB round trip, busy keys become a bottleneck. The brief allows gaps but never a duplicate, and that is how Postgres sequences behave: `nextval` never blocks other callers, never rolls back, and never returns the same value twice. You

can get gaps, but never duplicates.

How it works

1. **One sequence per counter key:** (template, city, period) for `[SEQ:n]`; (named counter, city, period) for `[SEQ_NAME:n]`. Templates with no `[CITY]` in the output (`PGR-`) get one global sequence. Names follow `seq_<counter>_<city>_<period>` in lower case, within Postgres's 63-character limit.
2. **Created at startup** for the current and next period, `START WITH` the configured `start` (1 by default). If another instance creates it at the same moment, catch `duplicate_object` and carry on.
3. **In memory:** `{next, end}` per key, guarded by a per-key lock. When `next == end`, one `nextval` returns `base`, and the block is `[base, base+100)`. Only one refill per key runs at a time; other callers wait for it.
4. **Bulk:** take what's left in the block. If more are needed, fetch several blocks in one trip with `SELECT nextval(…) FROM generate_series(1, k)`.
5. **Recent-IDs view:** an in-memory list of the last 50 IDs per (template, city) on each instance.

```
CREATE SEQUENCE IF NOT EXISTS seq_pg_receipt_amr_2026_27 START WITH 1
INCREMENT BY 100;
CREATE SEQUENCE IF NOT EXISTS seq_ws_amr                 START WITH 1000
INCREMENT BY 10;
CREATE SEQUENCE IF NOT EXISTS seq_pgr_complaint          START WITH 1
INCREMENT BY 100;

SELECT nextval('seq_pg_receipt_amr_2026_27');                       -- one
block
SELECT nextval('seq_pg_receipt_amr_2026_27') FROM generate_series(1, 5);  --
five blocks, one trip
```

Why it can't duplicate. Each `nextval` returns a different starting number, `INCREMENT` apart, so blocks never overlap, whether across instances, restarts or crashes. The in-memory lock only protects a block that this one instance already owns.

Why no tables at all. The only state that has to be durable and shared is "which numbers are taken", and a sequence holds exactly that. Templates and cities are configuration loaded from `data/` at startup, not data the service owns. The recent-IDs view is a convenience and can live in memory. Any table would either duplicate what the sequence

already guarantees (a counters table) or put a write back on every request (an issued-IDs log). If an audit trail is needed later, add an `issued_ids` table written in batches off the request path. That is an additive change and doesn't affect how numbers are allocated.

Option	DB trips per ID	Tables	Verdict
Counters table, `UPDATE` per request	1	1	Hot row lock; most moving parts
Counters table, hi-lo block	1/100	1	Works, but rebuilds what a sequence already is
Sequence with `CACHE 100`	1	0	Caches inside each Postgres connection; the app still makes a trip per `nextval`
Sequence `INCREMENT BY 100` + app block (chosen)	1/100	0	Fewest trips, no locks in the DB, nothing to maintain

Consequences

- **Gaps:** an instance restart or a new period leaves up to 99 unused numbers per key per instance. The brief allows this.
- **Order:** IDs are unique but not in strict time order across instances (A issues 1–100 while B issues 101–200).
- **Block size must come from the sequence:** at startup the app reads `increment_by` from `pg_sequences`, not from its own config. If an instance assumed 500 while the sequence increments by 100, its blocks would overlap.
- **Changing block size:** raising it (`ALTER SEQUENCE … INCREMENT BY 500`) is safe. Lowering it is not: first `setval` past the highest block already handed out, or blocks will overlap.
- **Narrow counters:** `SEQ_WS:4` has only 9,000 values, so it uses `INCREMENT BY 10`.
- **No issued-ID log:** there's no audit trail and no DB-level unique key as a backup. Uniqueness rests on the sequence plus correct rendering, and the load test is what proves it. The recent-IDs view is per instance and resets on restart. Audit history is out of scope.

How we prove it: run the concurrency test with `docker compose up --scale backend=3` behind Nginx, kill one backend container mid-run, and collect every ID the workers received. Zero duplicates is expected.

With another day: block size that adapts to load. A fixed 100 is too small for a hot key and too wasteful for a quiet one. The next step is to size each key's block per instance from its recent demand.

1. **Fixed small sequence step, variable fetch.** Keep every sequence at a small, fixed `INCREMENT BY` (e.g. 10) and vary only how many steps are fetched in one trip: `SELECT nextval(…) FROM generate_series(1, k)`. Because the sequence itself never changes, nothing can overlap. That's why this beats `ALTER SEQUENCE`, which is unsafe to lower.
2. **Pick k from demand.** Track an exponentially weighted rate of IDs/sec per key, and choose k so a block lasts about a target interval (e.g. 1–2 s), within `[1, max]`. A hot key like `PG_RECEIPT|BLR` grows to large blocks; a quiet key like `BPA_PERMIT|MDU` drops to one step of 10.
3. **Respect narrow counters.** Cap k from the remaining width space, so `SEQ_WS:4` never spends its 9,000 values on gaps.
4. **What it buys.** Hot keys make fewer DB trips; quiet keys and pods that scale down leave smaller gaps. Expose the current k and refill rate per key as metrics, so the tuning can be seen during the scaling test (ADR-002).

ADR-002: Build the service on Quarkus (JVM now, native later)

Decision: the backend is a Quarkus app, run in JVM mode for this submission. Native image is the first item for "another day". Accepted, 13 Nov 2026.

Context. The ID service is stateless apart from its in-memory blocks (ADR-001), so it scales by adding pods. How useful that is depends on how fast a new pod is ready to serve: slow startup means a traffic spike is over before the new pods arrive. Quarkus does most framework wiring at build time rather than at startup, so it starts fast and uses little memory.

Why JVM mode for the 3-hour POC

- Startup is typically about a second in JVM mode. Millisecond startup comes from **native** mode, which is roughly tens of ms (approximate, from Quarkus's published figures).

- A native build takes several minutes and needs several GB of RAM for the GraalVM/Mandrel builder. If that happened inside `docker compose up` on a reviewer's laptop, the build itself would be the biggest risk.
- The code is the same either way, so going native later is a build flag (`-Dnative`), not a rewrite.

Shape of the app

- **REST:** `quarkus-rest` with `POST /api/ids` (single) and `POST /api/ids/bulk`, plus one exception mapper for the error format.
- **Postgres:** `quarkus-reactive-pg-client` (or Agroal JDBC) calling only `CREATE SEQUENCE` at startup and `nextval` on refill. No Hibernate/ORM, since there are no tables.
- **Blocks:** `ConcurrentHashMap<String, Block>` with one lock per key. Single issues take the next value; bulk issues take a range in one go.
- **Config:** `templates.json` and `cities.csv` are read once at startup (`@Startup`). A config error fails startup with a clear message.
- **UI:** a static HTML/JS page served from `META-INF/resources`, so there's no separate frontend container.
- **Health:** `quarkus-smallrye-health` (`/q/health/ready`) feeds the compose healthcheck and, later, the Kubernetes readiness probe.

Consequences

- **Scaling interacts with ADR-001:** every new pod takes fresh blocks, and every pod removed abandons its unused ones. A sequence can't take numbers back, so frequent scale-up and scale-down means more gaps. Uniqueness is unaffected; larger gaps is the cost of elasticity, and it's within the brief.
- **Pod readiness is more than startup time:** image pull, scheduling and the first DB connection add seconds. Native shrinks the app's share of that, and a small native image also pulls faster.
- **Reviewers need nothing extra:** the Dockerfile does a multi-stage Maven build, so `docker compose up` is still the only command. The first build downloads dependencies and takes a few minutes; say so in the README.

With another day: prove elastic scaling with native

1. Build the native image (`./mvnw package -Dnative -Dquarkus.native.container-build=true`) and record its startup time and RSS against JVM mode.

2. Deploy to a local Kubernetes cluster (kind/k3d) with a HorizontalPodAutoscaler on CPU or request rate.
3. Run the concurrency test as a ramping load: scale from 1 to N pods and back during the run.
4. Report the time from scale-out decision to a ready pod, and that the full ID set still has zero duplicates across every pod that came and went.

ADR-003: Counter scoping and multi-tenancy

Decision: tenancy is a namespace in the sequence name, inside one shared Postgres database. A counter is scoped by exactly the variable parts that appear in the ID it produces, so `PGR-` gets one global counter while every other template gets one per city. Accepted, 13 Nov 2026.

Context. The brief scopes `[SEQ:n]` per (template, city) and named counters per city. Applied literally, that gives a duplicate for `PGR-[SEQ:8]`, whose output has no city. The email also asks how the design handles multi-tenancy.

The scoping rule: a part of the key that isn't printed in the ID can't be used to tell IDs apart, so it must not scope the counter.

Template shape	Counter key	Example sequence
`[SEQ:n]` with `[CITY]`	template + city (+ period if printed)	`seq_pg_receipt_amr_2026_27`
`[SEQ:n]` without `[CITY]`	template only (+ period if printed)	`seq_pgr_complaint`
`[SEQ_NAME:n]` with `[CITY]`	named counter + city (+ period if printed); shared by every template that uses it	`seq_ws_amr`
`[SEQ_NAME:n]` with no `counters` entry	rejected: 422 `COUNTER_NOT_CONFIGURED`	`EST_ALLOTMENT`

Tenant resolution. `city` accepts the tenant ID (`pb.amritsar`) or the city code (`AMR`), case-insensitively, against `cities.csv`. An unknown value returns 404 `UNKNOWN_CITY`. The response echoes the canonical `tenantId` and `cityCode`.

Isolation options

Option	Isolation	Cost	Verdict
Shared DB, sequences namespaced by name (chosen)	Independent numbering per tenant; no shared lock	None: a sequence holds no tenant data	Right for 10 cities and counters-only state
Schema per tenant	Separate namespaces	Onboarding and migrations per tenant	Adds work with no benefit here
Database per tenant/state	Hard separation, separate failures	Connection pools and operations × N	Only if a state requires data residency

Consequences

- **No noisy neighbours:** each tenant has its own sequences and its own in-memory lock per key, so a busy city never slows another one.
- **Onboarding a city** = a row in `cities.csv`. Its sequences are created at the next startup, or on first use.
- **Shared fate:** one Postgres serves every tenant, so an outage hits all of them. That's acceptable for the POC; split by state if residency or blast radius matters.
- **No caller authentication:** any caller can issue IDs for any tenant. Another day: take the tenant from a verified token claim, not the request body.

Configuration: data files as-is, settings in `application.yaml`. `cities.csv` and `templates.json` stay in `data/` exactly as provided and are read at startup. They are not copied into a table or into YAML.

Option	Verdict
Read `data/` files at startup (chosen)	Matches the brief ("use the provided data files as-is", "load both files at startup"). One source of truth.
Copy into `application.yaml`	A second copy of the data: a reviewer who edits `templates.json` would see no effect.
Load into a `cities` / `templates` table	Brings back a table for data the service doesn't own (ADR-001), plus a loader to maintain.

`application.yaml` holds only the service's own settings, including where to find the files:

```
idgen:
  data:
    templates-path: /data/templates.json
    cities-path: /data/cities.csv
  block-size: 100
  bulk-max: 1000
  zone: Asia/Kolkata
```

- **Compose** mounts `./data:/data:ro`, so a changed file takes effect on restart, with no rebuild.
- **The image** also copies `data/` in as a default, so the Docker Hub image runs on its own.
- **Invalid or missing files** stop startup with a clear message naming the file and the line, not a stack trace.
- **Another day:** if templates and cities must change at runtime, they belong in a master-data service with an admin API (DIGIT's MDMS plays this role), not in a table inside the ID generator.

ADR-004: Nginx as the load balancer, not an API gateway

Decision: a plain Nginx reverse proxy is the only public entry point. It round-robins across the backend replicas and does nothing else. No API gateway (Kong, Spring Cloud Gateway, a cloud API gateway). Accepted, 13 Nov 2026.

Context. The POC needs exactly two things in front of the backend: one public port, so reviewers' requests spread over 3 instances (ADR-001's multi-instance proof), and a way to keep serving while one instance restarts. What an API gateway adds (authentication, API keys, per-tenant rate limits, request transformation, a developer portal) is all a non-goal here: there is no auth in scope, and the API is two `POST`s and three `GET`s.

Option	What it brings	Cost here	Verdict
Nginx reverse proxy (chosen)	Round-robin, failover to the next instance, re-resolving replica addresses	One container, about 20 lines of config	Does exactly the job, nothing to learn or configure

Option	What it brings	Cost here	Verdict
Kong / Tyk	Auth, rate limits, keys, plugins	Its own DB or declarative config; plugins we would not turn on	Right once auth arrives, not before
Spring Cloud Gateway	Routing and filters in Java	A second JVM app to build, test and start	More code than the service it fronts
Traefik	Discovers containers by Docker labels	Mounts the Docker socket into a public container	Works, but adds a security trade-off for no gain
Envoy	Rich load balancing, active health checks	Verbose config, harder to read in a review	Overkill for 3 replicas
Cloud API gateway (AWS, Azure, GCP)	Managed auth, quotas, TLS	Can't run in `docker compose up` on a reviewer's laptop	Breaks the one-command rule

Consequences

- **No edge protection:** any caller can issue IDs for any tenant, with no rate limit. Acceptable because authentication is out of scope (ADR-003 already lists it as "another day").
- **Passive health only:** open-source Nginx removes an instance only after a request to it fails, then retries on the next one. A `POST` that already reached an instance is not retried, so a retry never issues IDs twice by accident (it would only burn numbers, but staying predictable matters more).
- **Re-resolution gap:** when replicas are recreated with new addresses, Nginx can return 502 for up to 5 seconds until it re-resolves `backend`. A plain restart keeps the address and is not affected.
- **Nothing in the app depends on Nginx.** Replacing it with a gateway or a Kubernetes Ingress later is a deployment change, not a code change.

Configuration (`nginx/nginx.conf`, mounted read-only into `nginx:alpine`):

```
# Docker's DNS returns one A record per backend replica; re-resolve every 5s
# so restarted/scaled replicas are picked up.
resolver 127.0.0.11 valid=5s ipv6=off;

upstream idgen_backend {
    zone idgen_backend 64k;          # shared memory, required for "resolve"
    server backend:8080 resolve;     # one entry per replica, round-robin
}

server {
    listen 8080;

    location / {
        proxy_pass http://idgen_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        add_header X-Upstream $upstream_addr always;   # which replica
answered

        # If an instance is down, try the next one. A POST that already
reached
        # an instance is never retried ("non_idempotent" is deliberately
absent).
        proxy_next_upstream error timeout http_502 http_503;
        proxy_connect_timeout 2s;

        # When no backend can answer (all down or restarting), reply in the
API's JSON error shape instead
        # of Nginx's HTML page. With proxy_intercept_errors off (the default)
this only covers errors Nginx
        # generates itself; a backend's own JSON error (e.g. 503
SEQUENCE_UNAVAILABLE) passes through as is.
        proxy_intercept_errors off;
        error_page 502 503 504 = @no_backend;
    }

    location @no_backend {
        default_type application/json;
        add_header Retry-After 5 always;
```

```
        return 503 '{"error":{"code":"SERVICE_UNAVAILABLE","message":"No
backend instance is available; try again shortly","details":{}}}';
    }
}
```

- **`resolver` + `resolve`:** Docker's DNS returns one address per replica, and Nginx re-checks every 5 s, so a recreated or scaled replica joins the pool without restarting Nginx.
- **`upstream`:** round-robin by default, so consecutive requests land on different replicas (the `X-Upstream` header shows which).
- **`proxy_next_upstream`:** a request that can't reach an instance, or gets 502/503 from one, moves to the next. A `POST` that was already sent is never retried, so a retry can't silently burn or double-issue numbers.
- **`error_page` → `@no_backend`:** when no backend can answer, Nginx replies `503 SERVICE_UNAVAILABLE` in the API's JSON error shape with `Retry-After: 5`, not its HTML page. It only covers errors Nginx generates; a backend's own JSON error passes through unchanged.
- **Port 8080** is the only one published on the host; Postgres and the backends stay internal.

With another day: put a real gateway in front once tenants authenticate.

- In production this service would sit behind the platform's existing gateway, which already does authentication and routing for the other city services.
- The gateway verifies the caller's token and passes the tenant as a trusted header or claim; the ID service stops trusting `city` from the request body (ADR-003).
- Add per-tenant rate limits at the gateway, so one city cannot burn through a narrow counter such as `SEQ_WS:4`.
- In Kubernetes, Nginx is replaced by a Service plus an Ingress (or the gateway itself), with readiness from `/q/health/ready` (ADR-002).

ADR-005: Templates and cities are loaded once at startup; changes need a restart

Decision: `templates.json` and `cities.csv` are read once at startup, validated, and held in memory (templates pre-parsed into tokens). Nothing re-reads them while the service runs, so adding or changing a city or template takes a restart of every backend instance. Accepted, 13 Nov 2026.

Context. The brief says to "load both files at startup" and treat them as configuration, not user input. Parsing once means no file I/O or string replacement per request (only rendering pre-parsed tokens), and a bad file is caught before the service takes traffic rather than halfway through a day.

What a restart does (verified by adding `ka.hubli,HBL,Hubli` to a copy of `cities.csv`):

- The instance logs `Loaded 6 templates and 11 cities` and creates the new city's sequences at startup (41 → 45); if Postgres is down then, they are created on first use.
- `PG-HBL-2026-27-000001` and `WS-HBL-1000-11-2026` issue immediately. `PGR_COMPLAINT` continues its one global counter.
- Instances not yet restarted answer `404 UNKNOWN_CITY` for the new city.

Which changes are safe

Change	Safe?	Why
Add a city	Yes	New city code → new sequences; no existing ID can repeat
Add a template	Yes	New template ID → new sequences
Add a named counter (e.g. `SEQ_EST_ALLOT`)	Yes	EST_ALLOTMENT starts issuing instead of 422
Remove a city or template	Yes for uniqueness	Requests for it fail; its sequences stay in Postgres, so re-adding it continues, never restarts
Change a counter's `start` after it has issued	No effect	The sequence already exists; `CREATE SEQUENCE IF NOT EXISTS` keeps the old start
Change an existing template's `format`	**Not safe in general**	If the counter key changes (e.g. `[CITY]` or a period removed, or the counter renamed), a new sequence starts from `start` and can re-issue IDs already printed under the old format. Give the new format a new template ID instead

Consequences

- **Restart to apply:** any change to `data/` needs every instance restarted. Compose mounts `./data:/data:ro`, so it is a restart, not a rebuild (`docker compose restart backend`).
- **Short-lived inconsistency during a rolling restart:** for a few seconds some instances know the new city and others answer 404. Uniqueness is unaffected; it just looks inconsistent.
- **All or nothing:** a bad row stops startup with one clear line naming the file and line, so an instance never runs on half a configuration.
- **No runtime validation of format edits:** the service cannot tell that a template's format changed since the last run, so the unsafe row above relies on the operator.

With another day: reload without a restart.

- **Reload in place:** an admin endpoint (or a file watcher) re-reads both files, validates them completely, and swaps the registries in one step. The registries already hold their maps in `volatile` fields and are replaced whole, so requests see either the old or the new configuration, never a mix. A failed validation keeps the old one and reports why.
- **Guard format changes:** keep the format each template ID was first seen with (a small `template_versions` table) and refuse a reload that changes the counter key of an existing template ID, pointing the operator to a new template ID.
- **Every instance at once:** publish a reload event (Postgres `LISTEN/NOTIFY` is enough) so all replicas switch together instead of one by one.
- **At platform scale:** cities and templates belong in a master-data service with an admin API and change events (DIGIT's MDMS plays this role), not in files inside the ID generator (ADR-003).

ADR-006: Number blocks and recent IDs stay in each instance's memory (no Redis)

Decision: each backend instance keeps two things in its own memory: its current block of numbers per counter (ADR-001) and its last 50 issued IDs per (template, city). Neither goes through Redis or any other central cache. We accept that IDs are not issued in strict sequence across instances, and that the recent-IDs view shows only what the instance answering has issued. Accepted, 13 Nov 2026.

Context. Nginx spreads requests round-robin over 3 instances (ADR-004), and each instance issues from its own block. Four calls in a row for Amritsar receipts return:

```
PG-AMR-2026-27-000001   instance A (block 1–100)
PG-AMR-2026-27-000101   instance B (block 101–200)
PG-AMR-2026-27-000201   instance C (block 201–300)
PG-AMR-2026-27-000002   instance A again
```

The brief asks for uniqueness, not order: "an occasional gap in the sequence is acceptable; a duplicate is never acceptable."

Why the numbers stay local, not in Redis. A shared counter (Redis `INCR`) would put a network round trip on every ID, which is exactly what the in-memory blocks avoid: today an instance goes to Postgres once per 100 IDs (once per 10 for narrow counters) and issues the rest from memory. Redis would also add a second stateful store to run, and one whose durability settings decide whether a counter can go backwards after a failure, which would mean duplicates. Measured with local blocks: 817 requests/s at p95 56 ms through Nginx (README, JMeter).

Why the recent view stays local for now. A shared view needs a write to a shared store on every issue (latency again) or an asynchronous pipeline (more moving parts). The brief asks for "a simple view of recently issued IDs", and it is a convenience, not an audit trail (ADR-001).

Option for the recent view	Shared across instances	Cost on the issue path	Verdict
Per-instance memory (chosen)	No	None	Simplest; list depends on which instance answers
Redis list per (template, city), `LPUSH` + `LTRIM 50`	Yes	A network write per request, unless sent asynchronously	Best "another day" option, off the request path
Postgres `issued_ids` table	Yes, and durable	A write per request, unless batched	Becomes the audit trail if one is ever needed (ADR-001)
Nginx sticky sessions (`ip_hash`)	No, but one browser always sees the same instance	None	Hides the effect for a single user; skews load across instances

Consequences

- **IDs jump by instance:** each instance counts up inside its own block, so IDs from different instances interleave (`000001` , `000101` , `000201` , `000002`). Sorting IDs does not give issue order; `issuedAt` does.
- **Gaps:** a restart leaves up to one block (100, or 10) unused per counter per instance (ADR-001). Uniqueness is unaffected.
- **Recent view differs per request:** refreshing can show another instance's list, and a restart empties that instance's list. The UI labels it "recently issued on this instance".
- **No extra infrastructure:** nothing beyond Postgres to run, and no dependency on Redis being up for issuing.

With another day: a shared recent view without slowing issuing.

- After an ID batch is issued, publish it **asynchronously** (not awaited by the request) to a Redis list per (template, city) with `LPUSH` + `LTRIM 0 49` , or to a Redis stream. The UI reads the recent view from Redis.
- Issue latency stays the same because the request doesn't wait for Redis; if Redis is down, the recent view goes stale but issuing carries on.
- Numbering stays on local blocks from Postgres sequences. Redis is never on the path that decides a number.

Implementation plan for Claude Code

Start 5:00 IST, submit by 8:00 IST. Code first; the documents are written from what was actually built. Save this document in the repo as `docs/architecture.md` , and point `CLAUDE.md` at it so every session starts with the same spec.

Rules for Claude Code (put these in `CLAUDE.md`)

- Java 25, Quarkus 3.x, Maven wrapper. Extensions: `rest` , `rest-jackson` , `reactive-pg-client` , `smallrye-health` , `hibernate-validator` . No ORM, no tables.
- Never modify anything in `data/` .
- Follow the API contract and error codes in this document exactly; DTOs are Java `record` s.
- Locks are per counter key, never global. Never call `nextval` per ID; always go through `BlockAllocator` . If endpoints run on virtual threads, use `ReentrantLock` instead of `synchronized` .
- Every step ends with `./mvnw test` green and `docker compose up --build` working. Commit after each step with a clear message.

- When unsure, choose the option that cannot produce a duplicate, and add a line to `docs/NOTES.md`.

Steps

1. **5:00–5:15 Skeleton.** Create the public GitHub repo, `.gitignore`, `docs/NOTES.md` and `CLAUDE.md`. Scaffold Quarkus, write the multi-stage `Dockerfile`, and write `docker-compose.yml` with Postgres + 3 backend replicas + Nginx. Add a `GET /api/ping`.
   - *Done when:* `docker compose up` serves `/api/ping` through Nginx on :8080 and `/q/health/ready` is UP.
2. **5:15–5:45 Config + templates.** `DataLoader`, `TemplateRegistry`, `CityRegistry`, `TemplateParser`, `CompiledTemplate.render`, `CounterKey`, plus unit tests.
   - *Done when:* render tests pass for all six templates, the FY boundary, padding growth, PGR global key and EST rejection; `GET /api/templates` and `GET /api/cities` return data.
3. **5:45–6:15 Allocation + API.** `SequenceStore`, `BlockAllocator`, `IdService`, `IdResource`, `ErrorMapper`, `RecentIds`, plus API tests.
   - *Done when:* single + bulk work for every template × city; every error code returns the documented shape.
4. **6:15–6:40 Concurrency test.** `loadtest/loadtest.py` + compose `loadtest` profile, plus the JMeter plan `loadtest/idgen.jmx` for throughput and latency.
   - *Done when:* 0 duplicates against 3 replicas, and again with `docker compose restart backend` mid-run. Save the output for the README.
5. **6:40–7:00 UI.** `index.html`: template + city dropdowns from the meta endpoints, a count field, issue, a results list, and a recent-IDs panel. **Code freeze at 7:00:** tag v1.0.0 and start the Docker Hub push in a second terminal.
6. **7:00–7:40 Documents from the code.** `spec.md` (problem, decisions table, out of scope), `docs/adr/0001-0006`, `plan.md` (milestones as they went, data model = sequences only, API from the real endpoints, test strategy, another day), `README.md` (`docker compose up`, the URL, test command, prebuilt image), `docs/ai-log.md` (kept up to date after every step; at the end add the exported session + 3–5 sentences from `docs/NOTES.md`).
7. **7:40–8:00 Verify and submit.** `git clone` into a new folder, `docker compose up`, run the load test, pull and run the Docker Hub image, then reply to the email with the repo link and the questions table.

If time runs short, cut in this order: the JMeter plan, then the 3 replicas + Nginx (run against one instance and list multi-instance under "another day"), then the recent-IDs panel, then the API tests. Hold the 7:00 code freeze. Never cut the load test or the fresh-clone check.

Repo and Docker image

One public GitHub repo holds the code, tests and documents. The image is also pushed to Docker Hub, but `docker compose up` always builds from source, so the reviewer's path never depends on the registry.

```
idgen/
├── docker-compose.yml        # postgres + backend (x3 option) + nginx
├── Dockerfile                # multi-stage: Maven build → JVM runtime
├── CLAUDE.md                 # rules for Claude Code; points to
docs/architecture.md
├── docs/NOTES.md             # running assumptions + AI mistakes (feeds
spec.md, docs/ai-log.md)
├── docs/architecture.md      # this document
├── nginx/nginx.conf          # round-robin to backend:8080
├── README.md                 # compose up, URL, test command, Docker Hub
image
├── egov-poc-id-gen.postman_collection.json   # Postman: every endpoint,
localhost:8080
├── spec.md  plan.md  docs/ai-log.md
├── docs/adr/0001-sequences-in-memory-blocks.md
├── docs/adr/0002-quarkus.md
├── docs/adr/0003-counter-scoping-multi-tenancy.md
├── docs/adr/0004-nginx-over-api-gateway.md
├── docs/adr/0005-config-loaded-at-startup.md
├── docs/adr/0006-per-instance-memory-no-redis.md
├── data/                     # templates.json, cities.csv — unchanged
├── src/main/java/…           # config, renderer, allocator, REST, errors
├── src/main/resources/application.yaml         # settings + data file paths
├── src/main/resources/META-INF/resources/index.html   # UI
├── src/test/java/…           # renderer unit tests
└── loadtest/                 # loadtest.py (duplicate check) + idgen.jmx
(JMeter throughput)
```

- **Before 5:00 (5 minutes):** create the Docker Hub repository (e.g. `nazaarblue/idgen`) and an access token, then `docker login`. Put nothing secret in the repo.

- **Commit at the end of every step** with a clear message. Combined with the code-first order, the git log becomes evidence of how you scoped the time.
- **Compose:** give the backend both `build: .` and `image: nazaarblue/idgen:1.0.0` and `pull_policy: build`. A clean checkout always builds from source and never pulls, and the built image carries the published name.
- **Push at the 7:00 freeze:** `docker buildx build --platform linux/amd64,linux/arm64 -t nazaarblue/idgen:1.0.0 -t nazaarblue/idgen:latest --push .`. Build for both architectures, because reviewers on Apple Silicon Macs need arm64. The arm64 leg runs under emulation and can take several minutes, so it runs while you write the docs.
- **README:** add a "Run the prebuilt image" section with `docker pull` and a `docker run` against the compose Postgres.
- **Another day:** a GitHub Actions workflow (`docker/build-push-action`) that pushes on every tag. Today a manual push avoids debugging CI against the clock.

Open questions and POC assumptions

The POC is built on the assumptions below rather than waiting for answers. They go into `spec.md`, and the table goes in the submission email as "question → what the POC assumes → easy to change if you prefer otherwise".

#	Question	What the POC assumes
1	The email asks for a spec + 2–3 ADRs + one API prototype; BRIEF.md asks for a full system. Which governs?	Built the full system from BRIEF.md. ADRs are in `docs/adr/`, and the issue endpoint is the "one API".
2	`PGR-[SEQ:8]` has no city in its output. Should its counter be global to keep IDs unique across cities?	One global sequence for PGR_COMPLAINT, which never resets.
3	Should counters reset when the FY / year / month in the ID changes?	Yes, but only when that period appears in the ID: a new period means a new sequence starting again from `start`.
4	When a sequence passes its padding width (e.g. `SEQ_WS:4` past 9999), should the ID grow or fail?	The ID grows wider and is never truncated. A warning is logged.

#	Question	What the POC assumes
5	Is IST the reference timezone for FY and date placeholders?	Asia/Kolkata, read once per request (bulk included).
6	Is there a maximum bulk `count`, and what load will the review use?	`count` 1–1000. Tested with 50 workers against 3 backend instances.
7	IDs come from per-instance blocks of 100. Is it acceptable that they aren't in strict time order and may have gaps of up to 99 after a restart?	Yes. The brief allows gaps; only uniqueness is guaranteed (ADR-001).
8	Does the recently-issued view need to survive restarts or show all instances?	No. It's an in-memory list per instance, with no tables (ADR-001).